

MongoDB

MongoDB is a document-oriented open-source NoSQL database. It is one of the most popular and widely used NoSQL databases.

MongoDB is an open-source, cross-platform, and distributed document-based database designed for ease of application development and scaling.

MongoDB name is derived from the word "Humongous" which means huge, enormous. MongoDB database is built to store a huge amount of data and also perform fast.

MongoDB is not a Relational Database Management System (RDBMS). It's called a "NoSQL" database. It is opposite to SQL based databases where it does not normalize data under schemas and tables where every table has a fixed structure. Instead, it stores data in the collections as JSON based documents and does not enforce schemas. It does not have tables, rows, and columns as other SQL (RDBMS) databases.

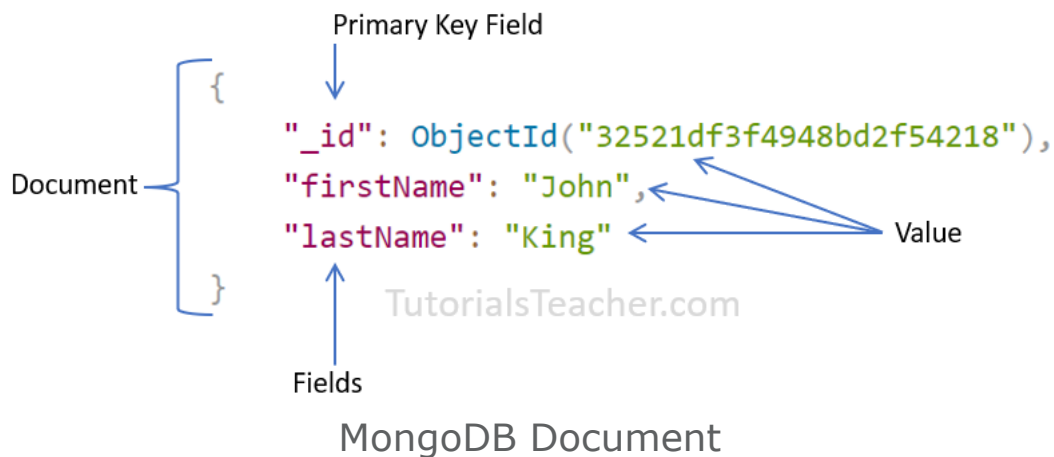
The following table lists the relation between MongoDB and RDBMS terminologies.

MongoDB (NoSQL Database)	RDBMS (SQL Server, Oracle, etc.)
Database	Database
Collection	Table
Document	Row (Record)
Field	Column

In the RDBMS database, a table can have multiple rows and columns. Similarly in MongoDB, a collection can have multiple documents which are equivalent to the rows. Each document has

multiple "fields" which are equivalent to the columns. Documents in a single collection can have different fields.

The following is an example of JSON based document.



Features of MongoDB

These are some important features of MongoDB:

1. Support ad hoc queries

In MongoDB, you can search by field, range query and it also supports regular expression searches.

2. Indexing

You can index any field in a document.

3. Replication

MongoDB supports Master Slave replication.

A master can perform Reads and Writes and a Slave copies data from the master and can only be used for reads or back up (not writes)

4. Duplication of data

MongoDB can run over multiple servers. The data is duplicated to keep the system up and also keep its running condition in case of hardware failure.

5. Load balancing

It has an automatic load balancing configuration because of data placed in shards.

Sharding is a method for **distributing a single dataset across multiple databases, which can then be stored on multiple machines**. This allows for larger datasets to be split into smaller chunks and stored in multiple data nodes, increasing the total storage capacity of the system.

6. Supports map reduce and aggregation tools.

7. Uses JavaScript

instead of Procedures.

While MongoDB does not offer stored procedures as traditional relational databases do, it provides multiple tools that you can use to achieve the same result. By using **aggregation pipelines** with the proper indexes, you can perform resource-intensive queries from the database server.

8. It is a schema-less database written in

A schema is a **collection of database objects like tables, triggers, stored procedures, etc.**

9. Provides high performance.

10. Stores files of any size easily without complicating your stack.

11. Easy to administer in the case of failures.

12. It also supports:

JSON data model with dynamic schemas

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident
occaecati excepturi optio reprehenderit",
```

```
"body": "quia et suscipit\nsuscipit recusandae  
consequuntur expedita et cum\nreprehenderit molestiae ut  
ut quas totam\nnostrum rerum est autem sunt rem eveniet  
architecto"  
}
```

Auto-sharding for horizontal scalability

Built in replication for high availability

Now a day many companies using MongoDB to create new types of applications, improve performance and availability.

NoSQL Databases

We know that MongoDB is a NoSQL Database, so it is very necessary to know about NoSQL Database to understand MongoDB thoroughly.

What is NoSQL Database

Databases can be divided in 3 types:

1. RDBMS (Relational Database Management System)
2. OLAP (Online Analytical Processing)
3. NoSQL (recently developed database)

NoSQL Database

NoSQL Database is used to refer a non-SQL or non relational database.

It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.

Advantages of NoSQL

- It supports query language.
- It provides fast performance.
- It provides horizontal scalability.

MongoDB advantages over RDBMS

In recent days, MongoDB is a new and popularly used database. It is a document based, non relational database provider.

Although it is 100 times faster than the traditional database but it is early to say that it will broadly replace the traditional RDBMS. But it may be very useful in term to gain performance and scalability.

A Relational database has a typical schema design that shows number of tables and the relationship between these tables, while in MongoDB there is no concept of relationship.

MongoDB Advantages

- **MongoDB is schema less.** It is a document database in which one collection holds different documents.
- There may be **difference between number of fields, content and size of the document** from one to other.
- **Structure of a single object is clear** in MongoDB.
- There are **no complex joins** in MongoDB.
- MongoDB provides the **facility of deep query** because it supports a powerful dynamic query on documents.
- It is very **easy to scale**.
- It **uses internal memory for storing working sets** and this is the reason of its fast access.

Distinctive features of MongoDB

- Easy to use
- Light Weight
- Extremely faster than RDBMS

Where MongoDB should be used

- Big and complex data
- Mobile and social infrastructure
- Content management and delivery
- User data management
- Data hub

Some Real-World Companies That Use MongoDB

1. eBay

eBay is a multinational company that provides a platform for the customer to customer sales. As an eCommerce platform, eBay has to deal with storing and managing huge amounts of customer data. Dealing with such a large volume of data can be a cumbersome task. However, MongoDB has helped streamline this task for eBay. The company uses MongoDB to make a search suggestion feature, making it easier for customers to find relevant products in less time. It is currently running a large number of projects in MongoDB like merchandising categorization, cloud management, metadata storage, search suggestions.

2. MetLife

MetLife is a leading company in employee benefit programs, annuities, and insurance. There are more than 90 million customers in the Middle East, Europe, Asia, Latin America, Japan, United States. MetLife is using MongoDB as a database engine for its advanced customer service application called The Wall.

This application provides a combined view of transactions, policy details, and other details of MetLife Customers. It looks like Wall of Facebook, which takes data from 70 legacy systems and merges it into a single record. It stores 24 Terabytes of data and runs across six servers in two data centers. MetLife is working on a series of Big Data projects where MongoDB-based applications are part of it. MetLife wall, created with the help of MongoDB, allows the company to have a 360-degree view of their customers, their policy details and transactions.

3. Shutterfly

Shutterfly is one of the most popular online photo sharing platforms, and it is using MongoDB to manage and store more than 6 billion images, which has a transaction rate of up to 10,000 operations per second. Shutterfly earlier used Oracle but later transitioned to MongoDB.

Shutterfly company realized that moving to a non-relational database will help them to increase their scalability, performance, and productivity of the programmer. The company decided to adopt a document database approach because of the large volumes of data involved.

Shutterfly did consider many other alternate database systems like BerkeleyDB, CouchDB, or Cassandra. The company has confirmed that they are pleased with their decision of transitioning from Oracle to MongoDB.

There were two main reasons for this transition – MongoDB allows flexibility with Schema and facilitates horizontal scalability. Shutterfly is using MongoDB Atlas as its database application. The business is running multiple Atlas clusters and efficiently managing thousands of transactions every minute.

The company also used Mongomirror by MongoDB for quick data migration. Mongomirror is a tool that facilitates manual data migration from a MongoDB replica set to a MongoDB Atlas replica set. MongoDB helps the business scale effortlessly during the peak seasons and cope with customers' demands

4. Aadhar

It is India's Unique Identification project, which has the biggest biometrics database in the world. Aadhar Project is using MongoDB as its database to store a massive amount of demographic and biometric data of more than 1.2 billion Indians. MongoDB is being used for the storage of images in the Aadhar project.

One of the major reasons why MongoDB was chosen to partner with the Aadhar project was that the database system facilitates capturing, processing, searching and analyzing a massive amount of unstructured datasets. Moreover, MongoDB is capable of storing large volumes of biometric data and images.

Performance analysis of MongoDB and RDBMS

- In relational database (RDBMS) tables are using as storing elements, while in MongoDB collection is used.
- In the RDBMS, we have multiple schema and in each schema we create tables to store data while, MongoDB is a document oriented database in which data is written in BSON format which is a JSON like format.
- MongoDB is almost 100 times faster than traditional database systems.

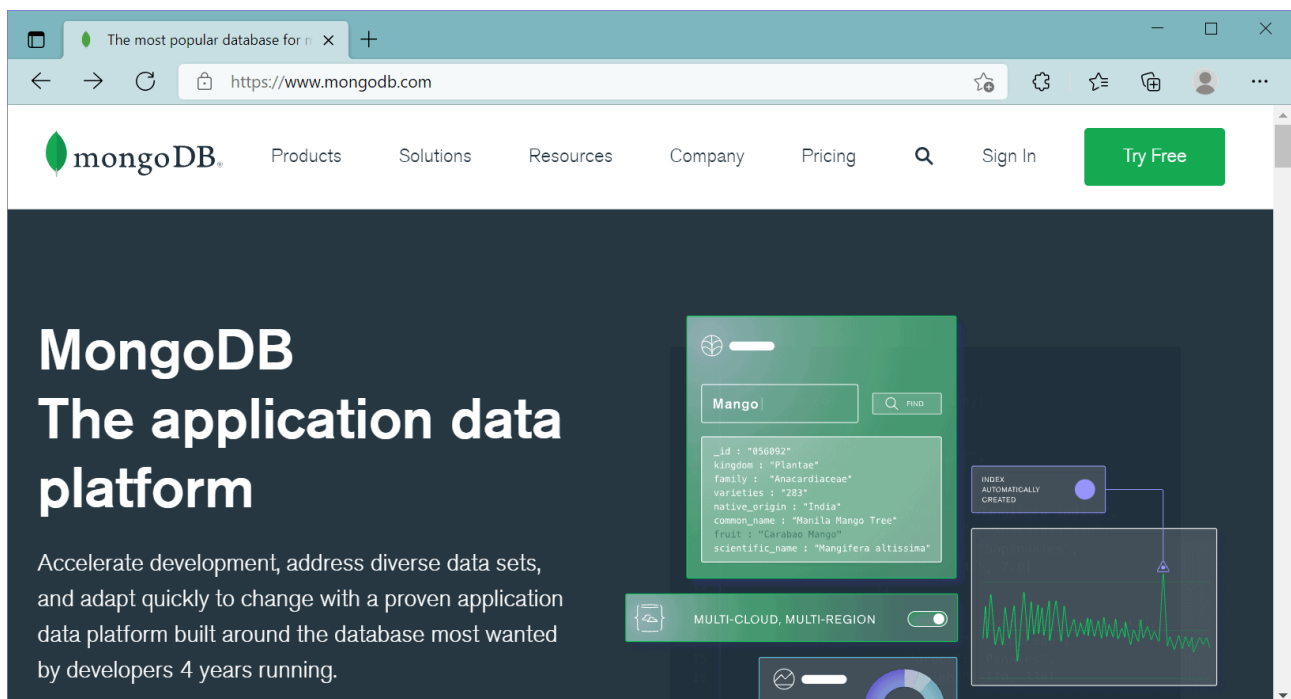
MongoDB Datatypes

Following is a list of usable data types in MongoDB.

Data Types	Description
String	String is the most commonly used datatype. It is used to store data. A string must be UTF 8 valid in mongodb.
Integer	Integer is used to store the numeric value. It can be 32 bit or 64 bit depending on the server you are using.
Boolean	This datatype is used to store boolean values. It just shows YES/NO values.
Double	Double datatype stores floating point values.
Min/Max Keys	This datatype compare a value against the lowest and highest bson elements.
Arrays	This datatype is used to store a list or multiple values into a single key.

Object	Object datatype is used for embedded documents.
Null	It is used to store null values.
Symbol	It is generally used for languages that use a specific type.
Date	This datatype stores the current date or time in unix time format. It makes you possible to specify your own date time by creating object of date and pass the value of date, month, year into it.

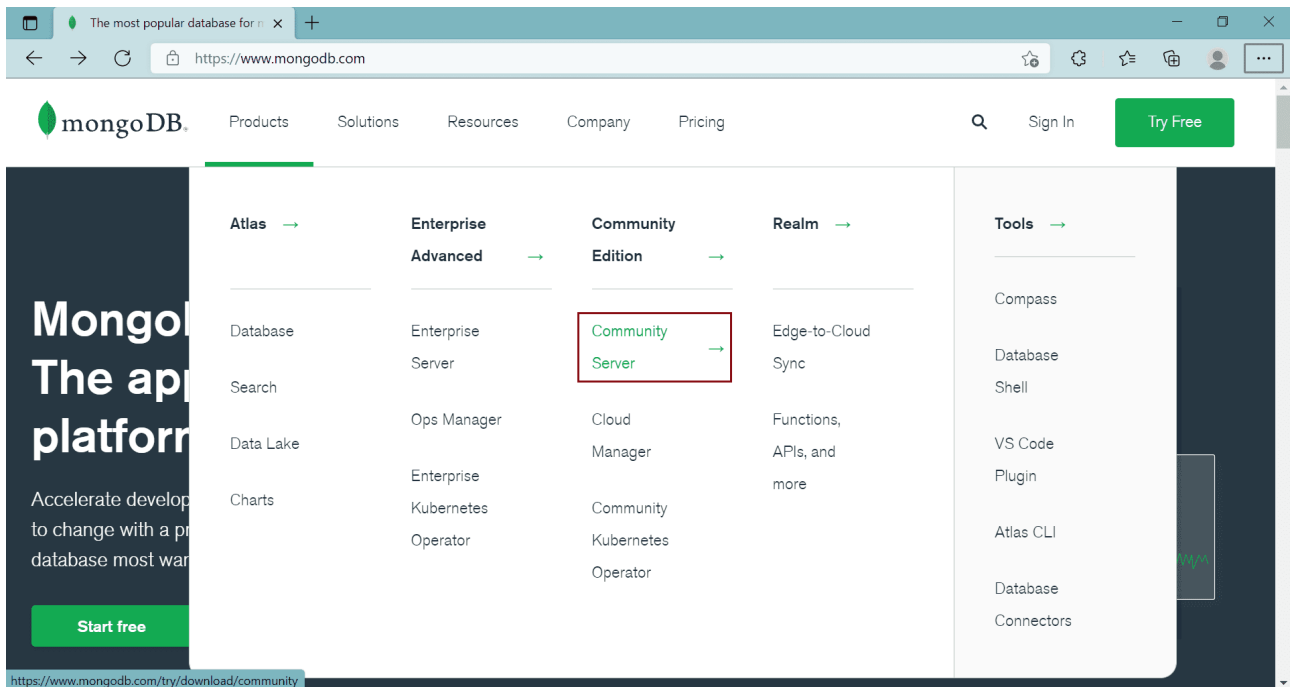
Install MongoDB Server, MongoDB Shell, Compass on Windows



Here you will learn how to install MongoDB server, MongoDB Shell, and compass on your local Windows machine.

Visit www.mongodb.com to download the MongoDB installer for your required platform.

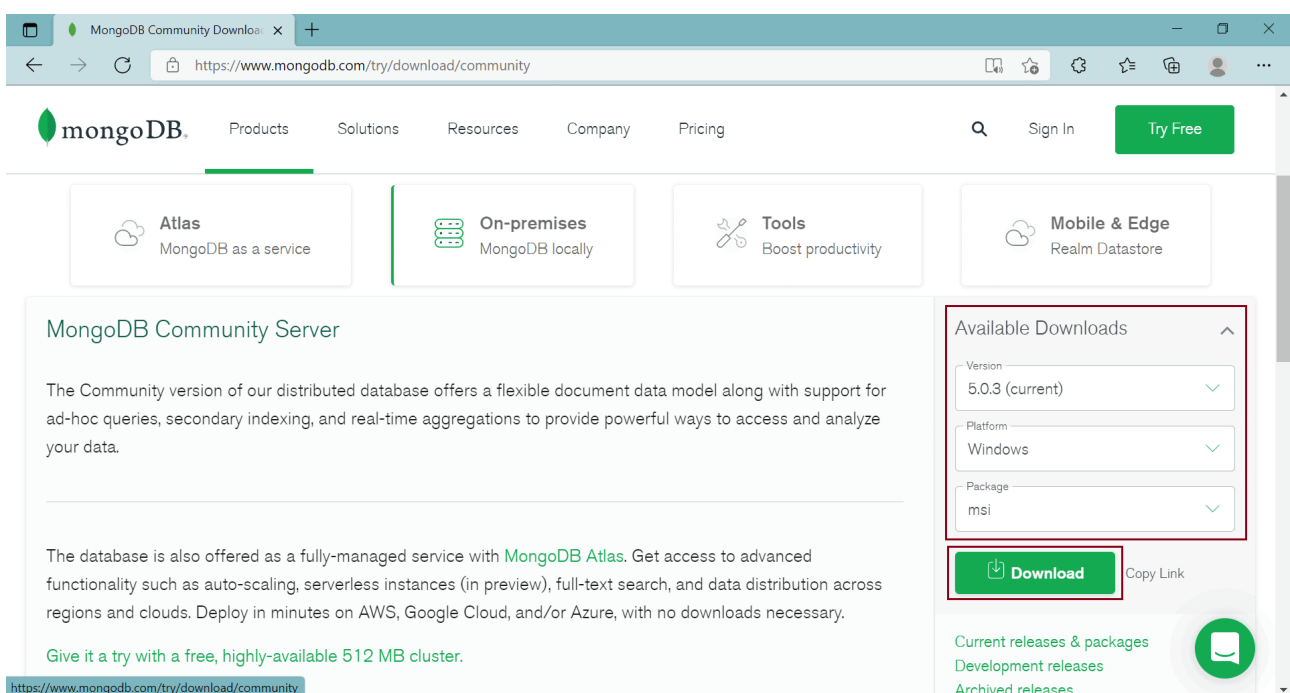
Here, we are going to install a free MongoDB database server on our local Windows machine. So, click on the Product menu -> Community Server, as shown below.



MongoDB Community Edition

This will open a download page where you can select the version, platform, and package options.

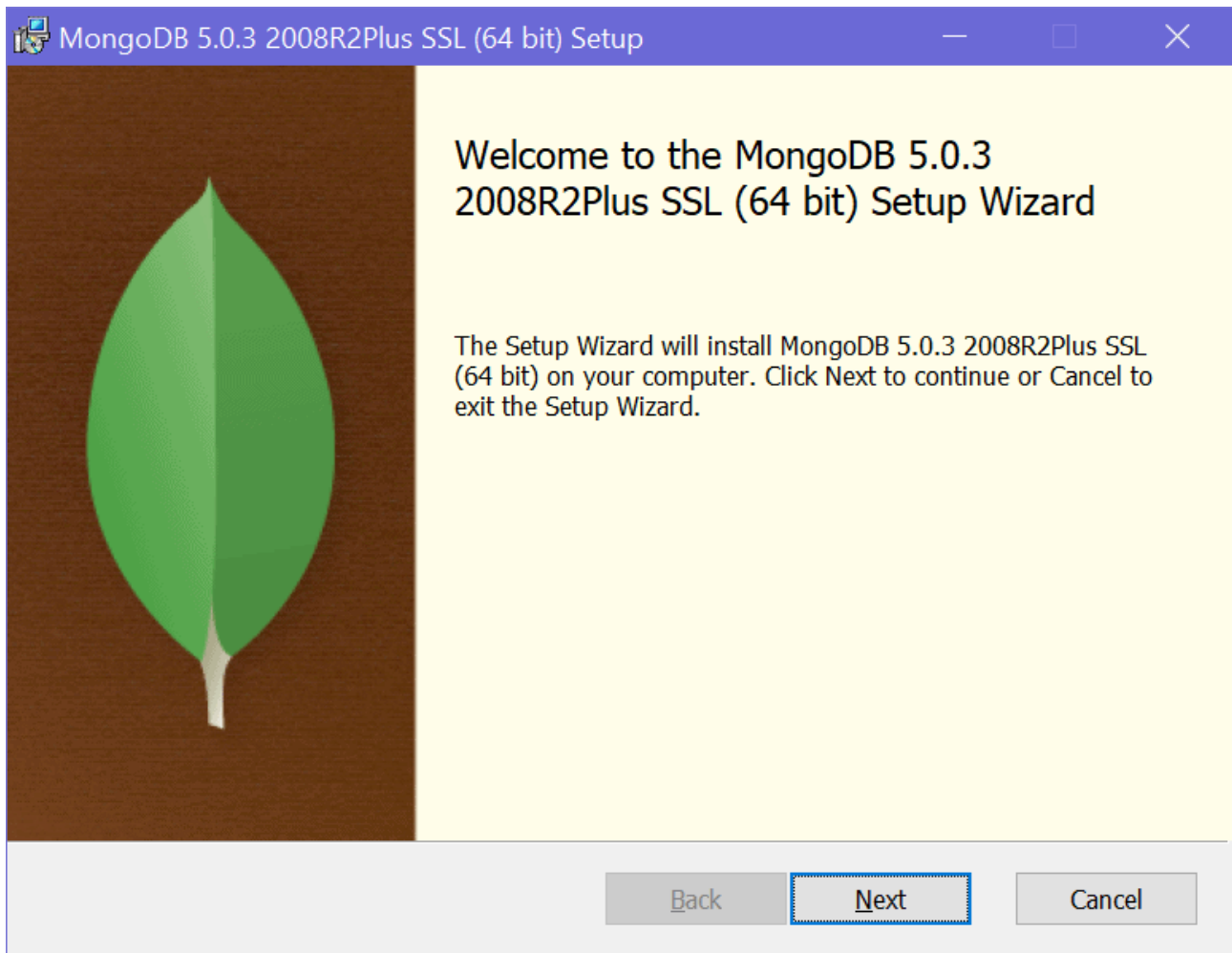
Here, we will download the latest version of MongoDB, Windows platform, and msi file as a package, as shown below.



Download MongoDB

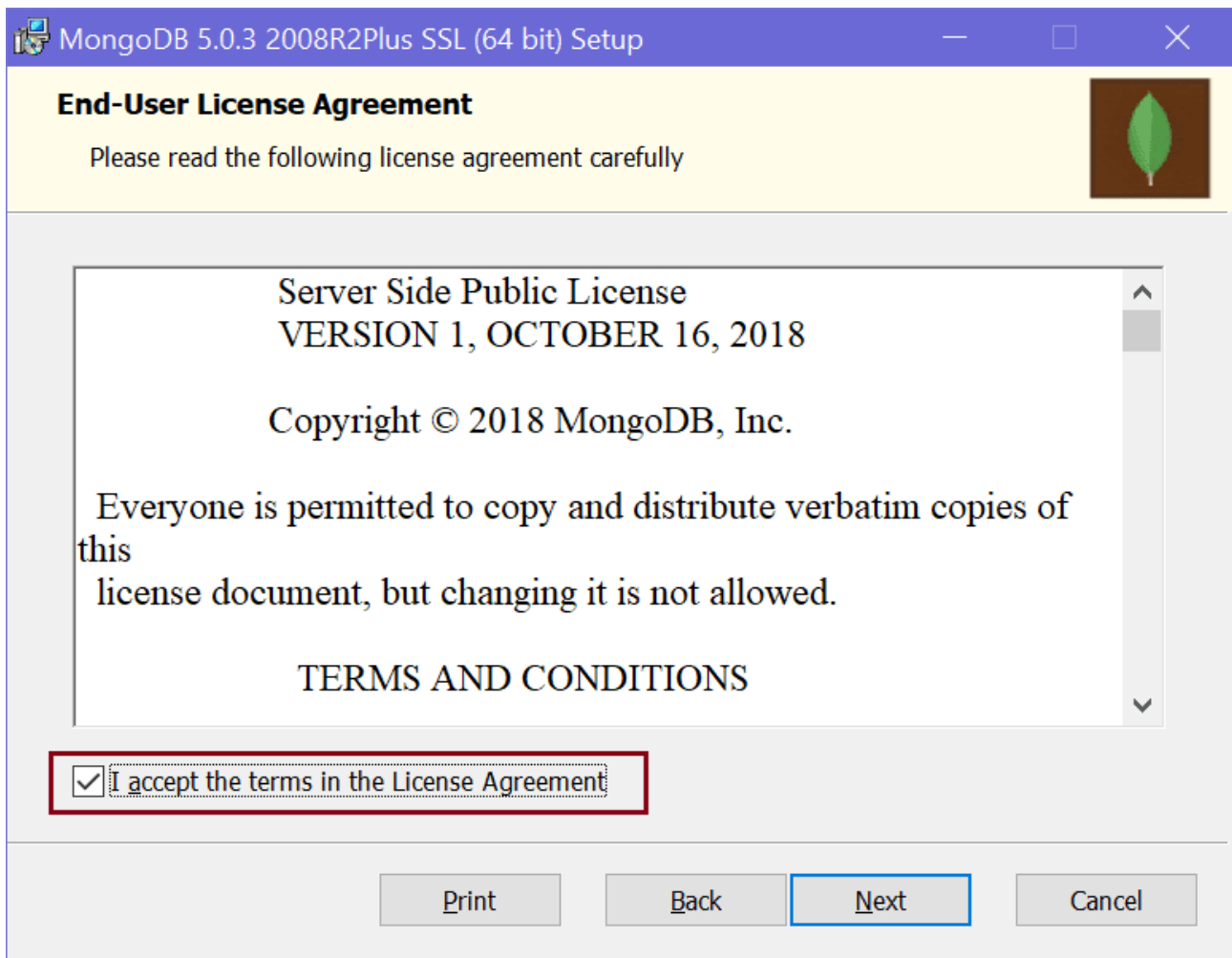
Click on the Download button to download the installer file.

Once fully downloaded, click on the msi file to start the installation wizard, as shown below.



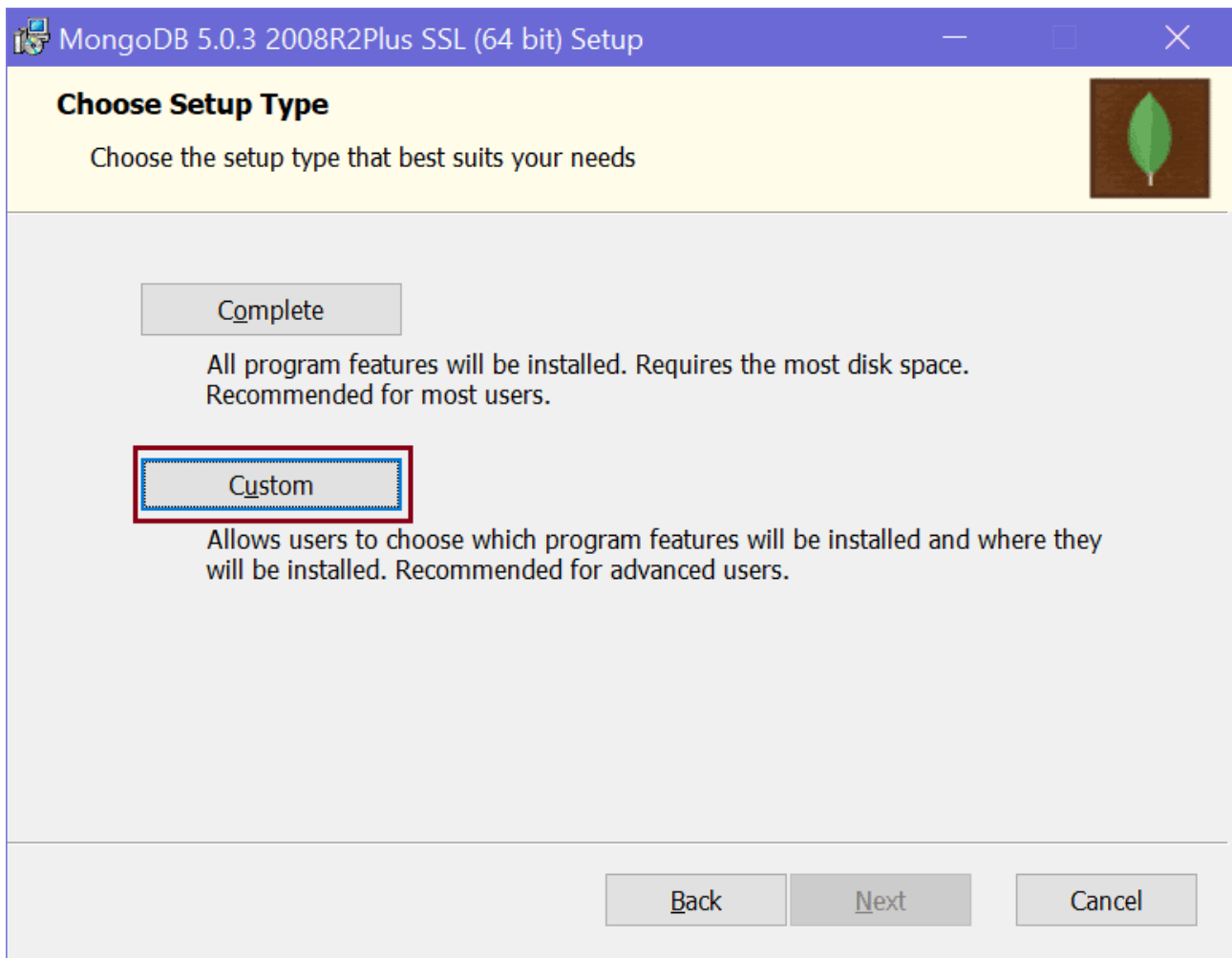
MongoDB Installation Wizard

Click Next to start the installation.



MongoDB Installation Wizard

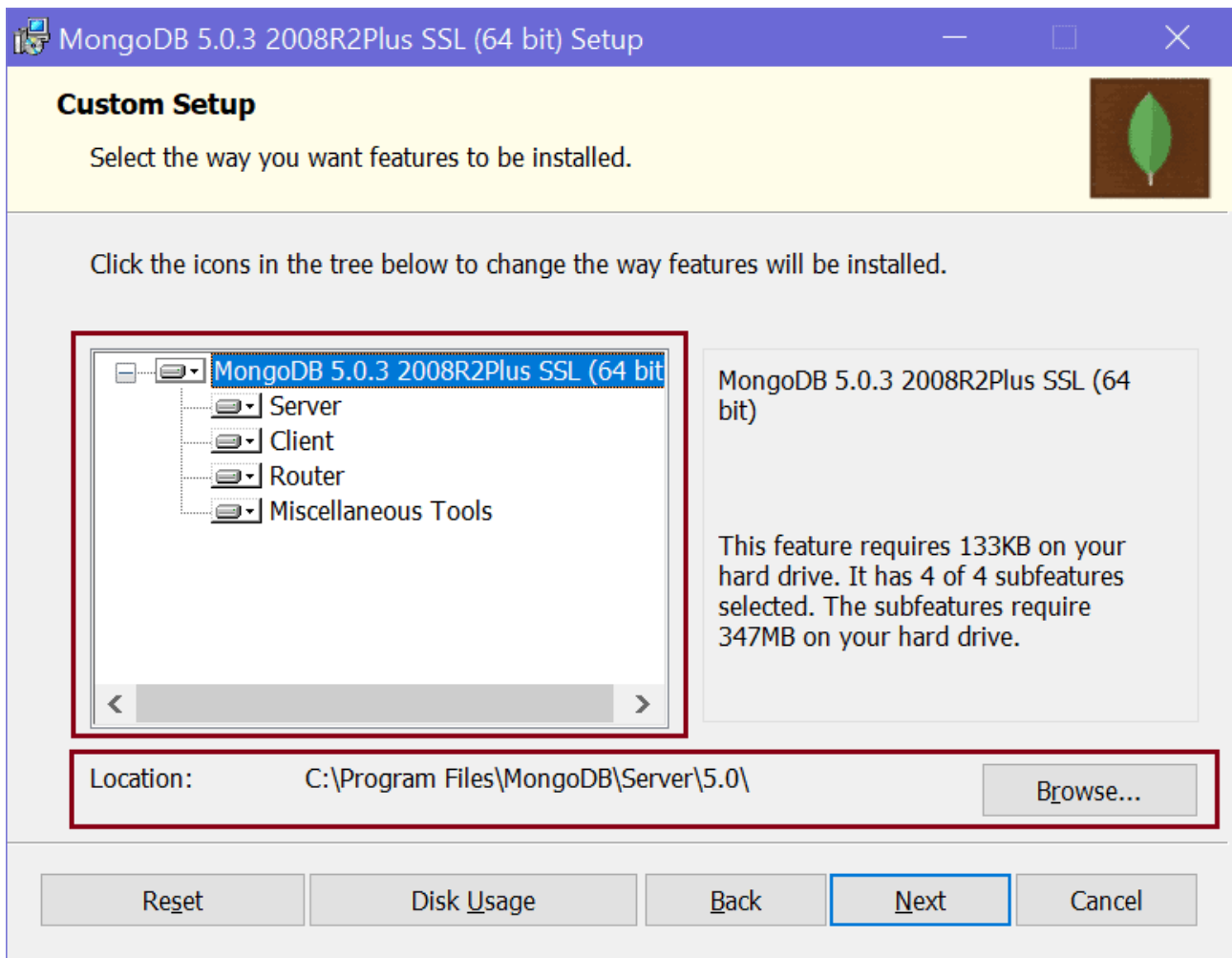
Check the "I accept the terms in the License Agreement" checkbox and click Next.



MongoDB Installation Wizard

Here, you will have two options for installation: Complete and Custom. The complete option will install all the features. The custom option allows you to select only required features.

You can select either of these two options. Here, we will select the custom option just to show you what all things it will install. So, click on the Custom option which will take you to the next step, as shown below.



MongoDB Installation Wizard

In the custom setup page, expand the MongoDB node to see what all features will be installed. It will install Server, Client, Router, and Miscellaneous Tools for the MongoDB database. It also displays the location where the MongoDB is going to be installed. You may change it or keep the default location and click Next.

Click Next to configure the MongoDB service, as shown below.

Service Configuration
Specify optional settings to configure MongoDB as a service.

☒ Install MongoD as a Service

☒ Run service as Network Service user

☐ Run service as a local or domain user:

Account Domain: .

Account Name: MongoDB

Account Password:

Service Name: MongoDB

Data Directory: C:\Program Files\MongoDB\Server\5.0\data\

Log Directory: C:\Program Files\MongoDB\Server\5.0\log\

< Back Next > Cancel

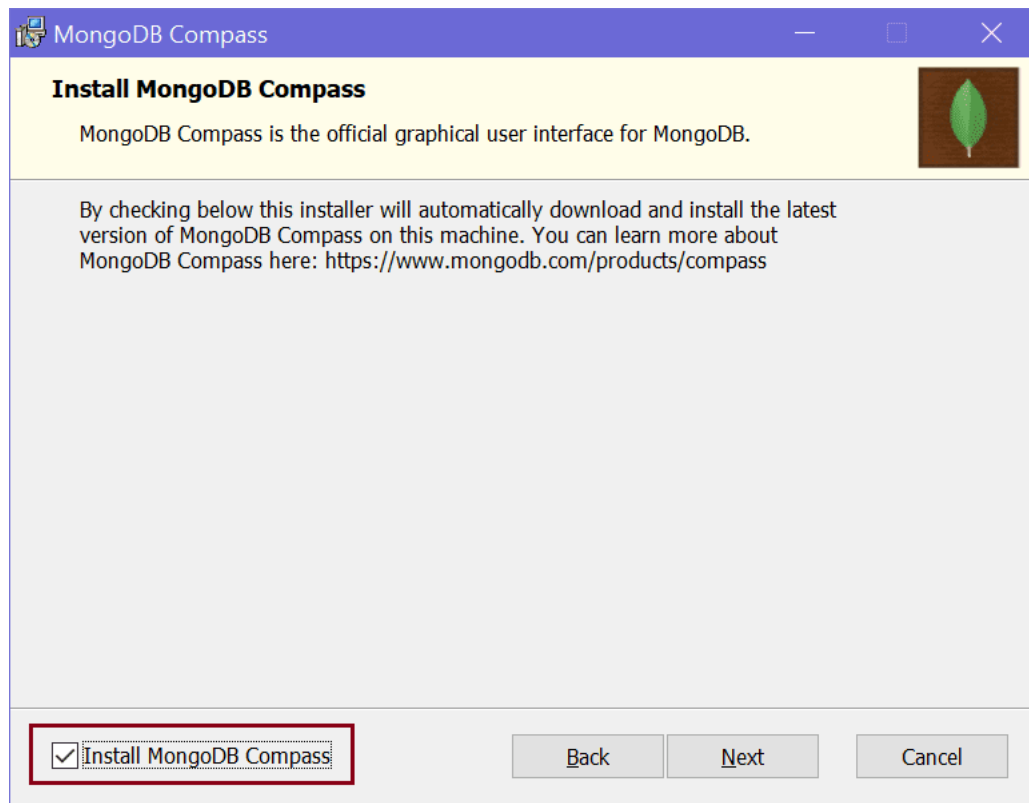
Configure MongoDB Service

MongoDB server will be installed as a service on your local Windows machine. As you can see above, you have an option to run a service as Network Service user or as a local or domain user. We will select "Run service as Network Service User" radio button.

You may change the default service name but it is recommended to keep the default "MongoDB" name to identify it easily.

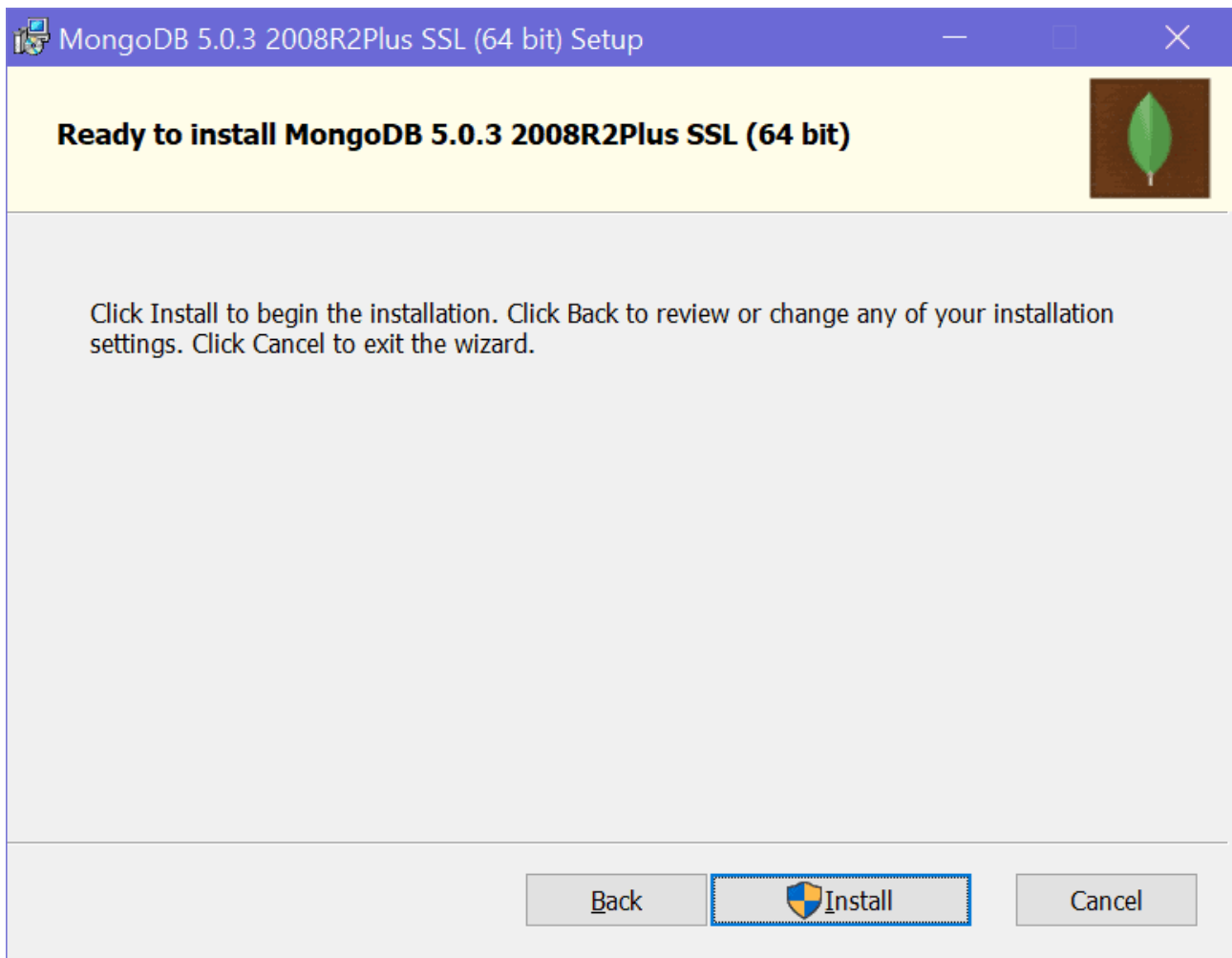
MongoDB database stores the data as BSON files on your local machine. You can change the default location where the data files and log files are going to be stored. Here, we will keep the default paths and click the Next button.

On the next page, select "Install MongoDB Compass" checkbox and click Next. MongoDB Compass is a GUI tool for MongoDB database where you can visually explore data, run queries, and optimize performance.



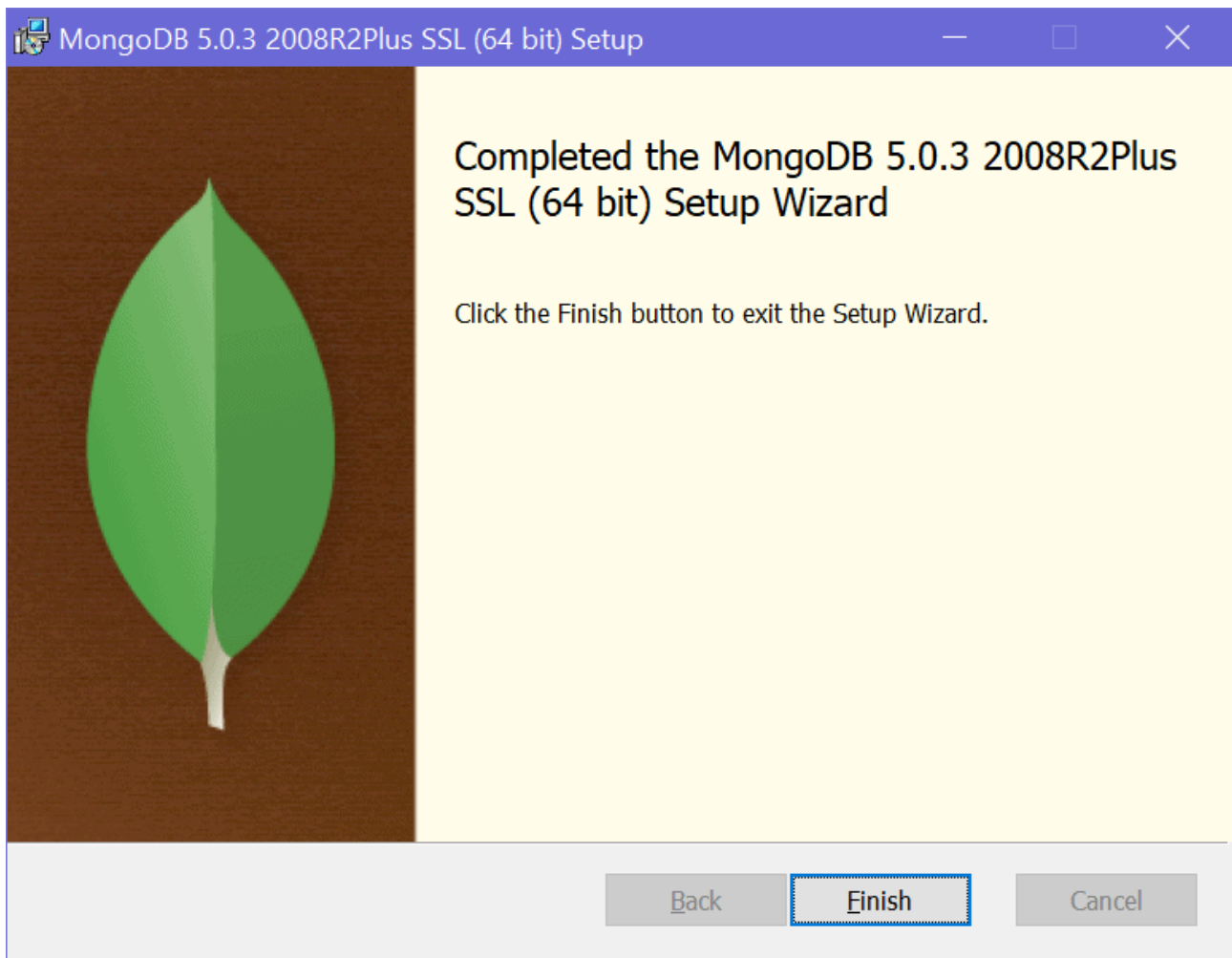
MongoDB Compass Installation

Next, click on the Install button to start the installation.



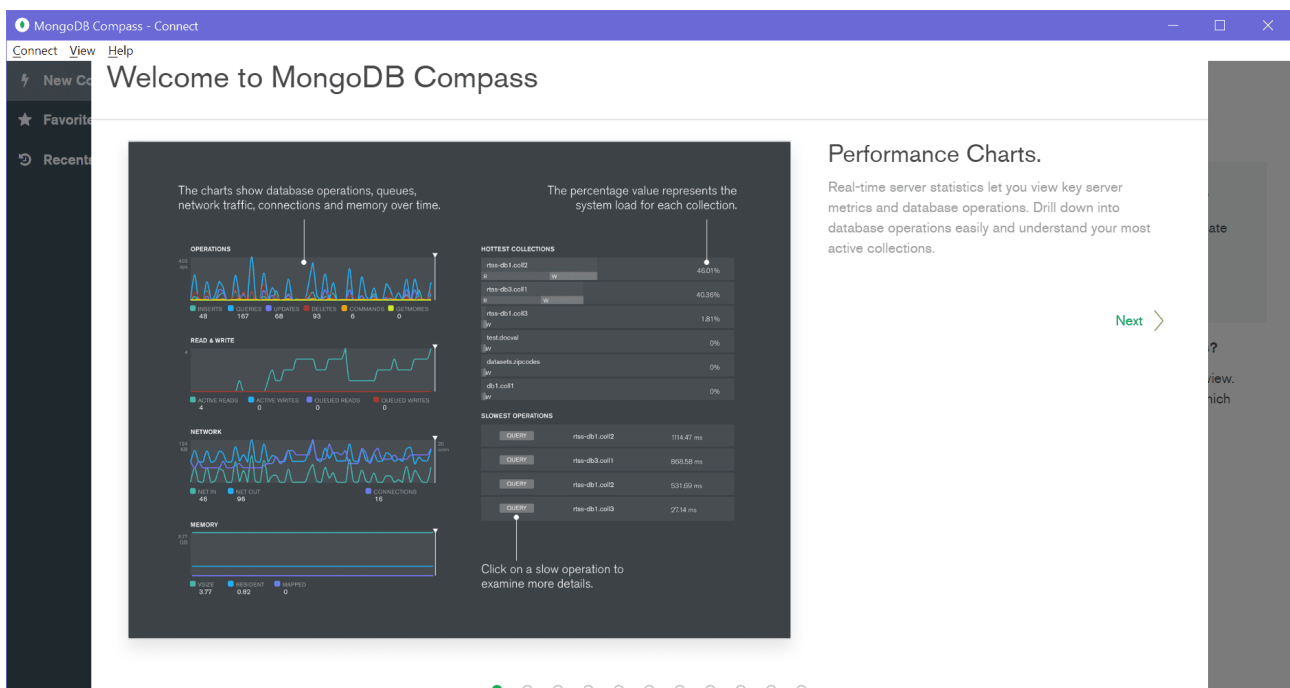
MongoDB Installation

It will take a few minutes to install. Once installed successfully, click on the Finish button to close the wizard.



MongoDB Installation

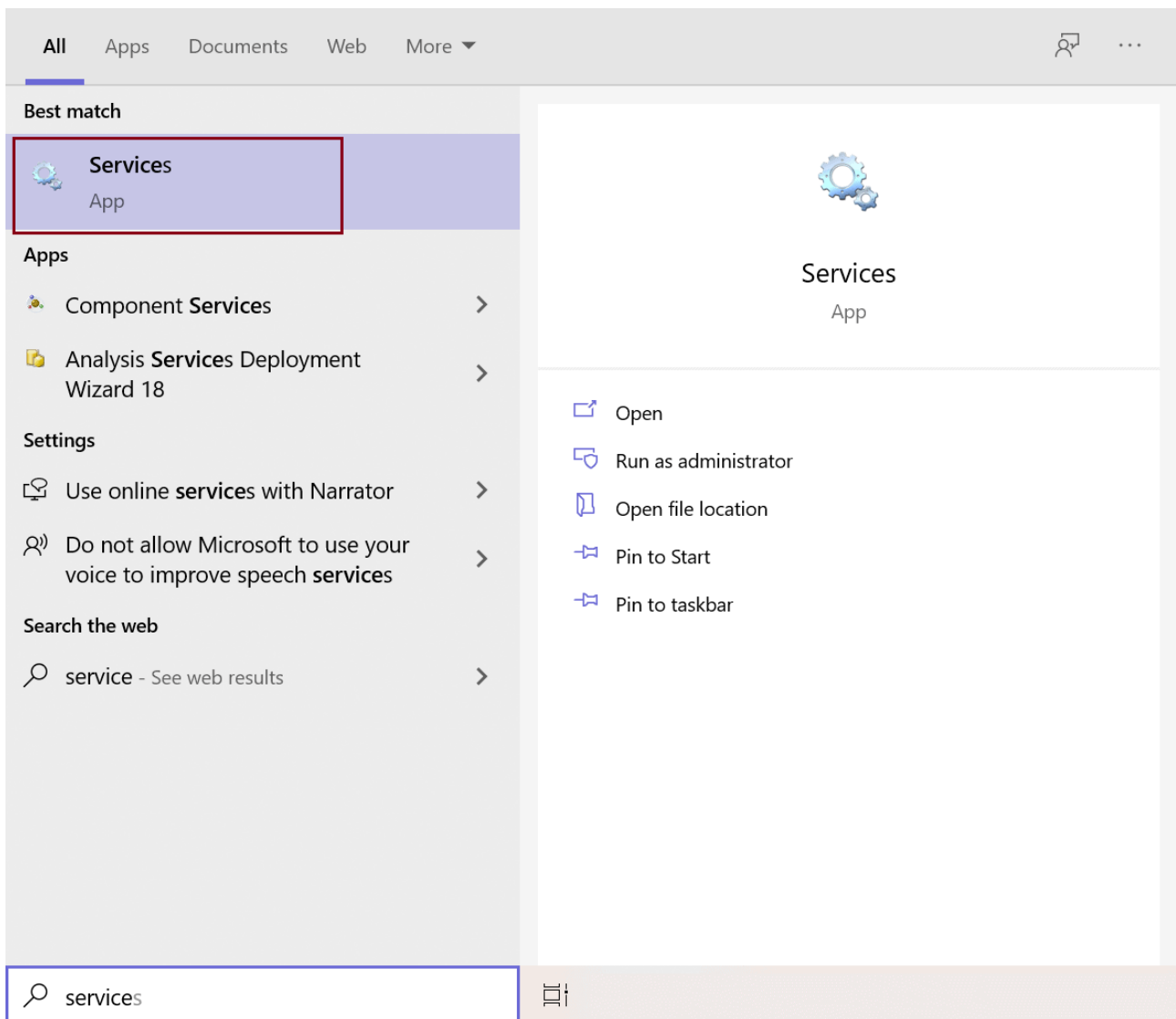
It will also open MongoDB Compass, as shown below.



MongoDB Compass

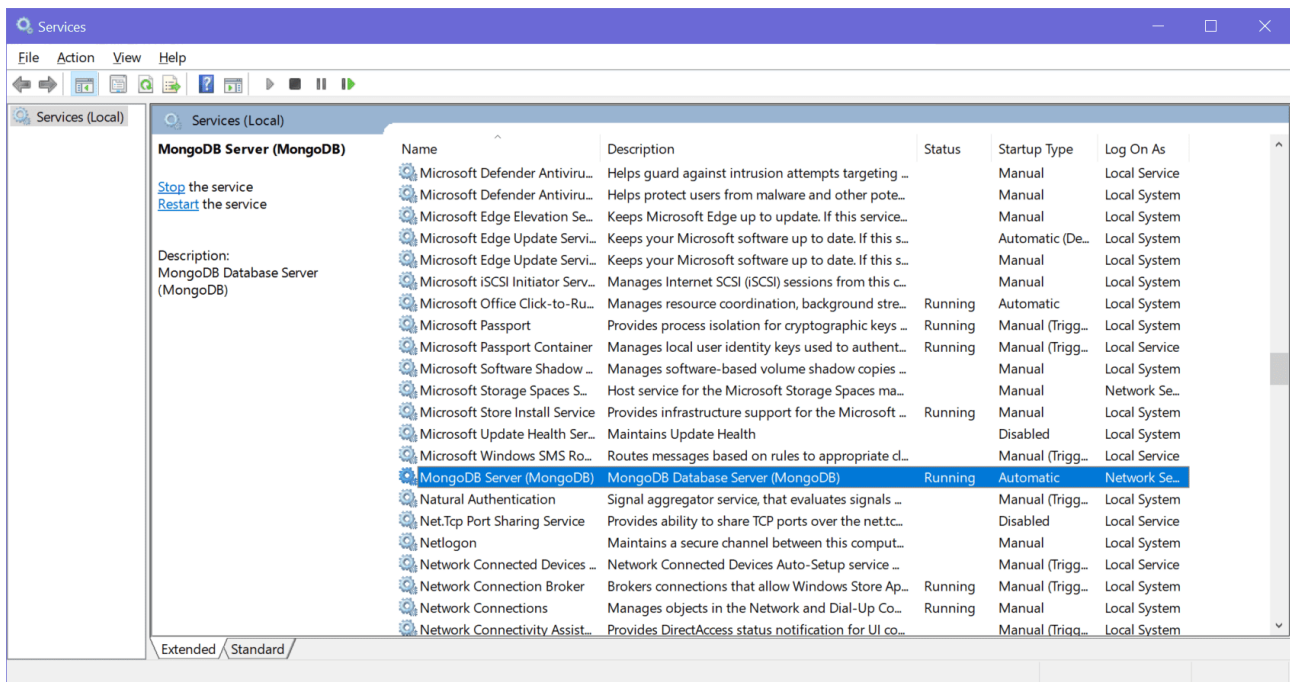
MongoDB Server

We installed MongoDB as a Network service. To see this, open Services by searching "service" in the windows search box and click on the Services app, as shown below.



MongoDB Service

On the Services window, navigate to MongoDB Server, as shown below. You will find that it is already up and running.



MongoDB Service on Windows

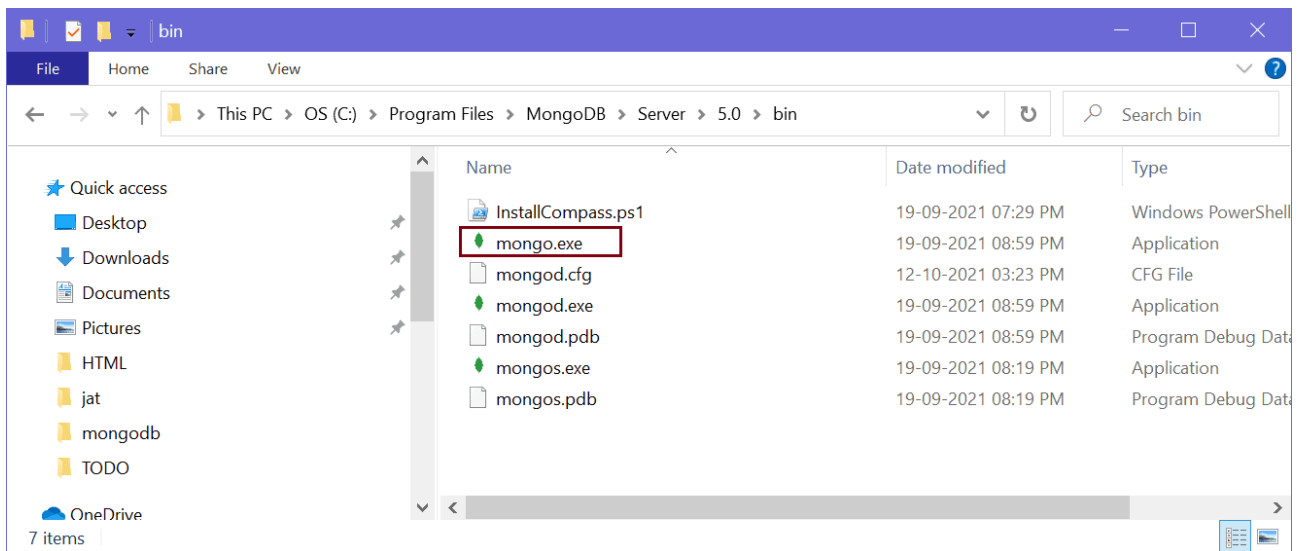
Now, the MongoDB Server service is already running, you can connect the MongoDB client to connect with this MongoDB server and execute the commands.

MongoDB Client

MongoDB clients can be your application, MongoDB Shell, MongoDB Compass, or anything which wants to connect and store data to the MongoDB server.

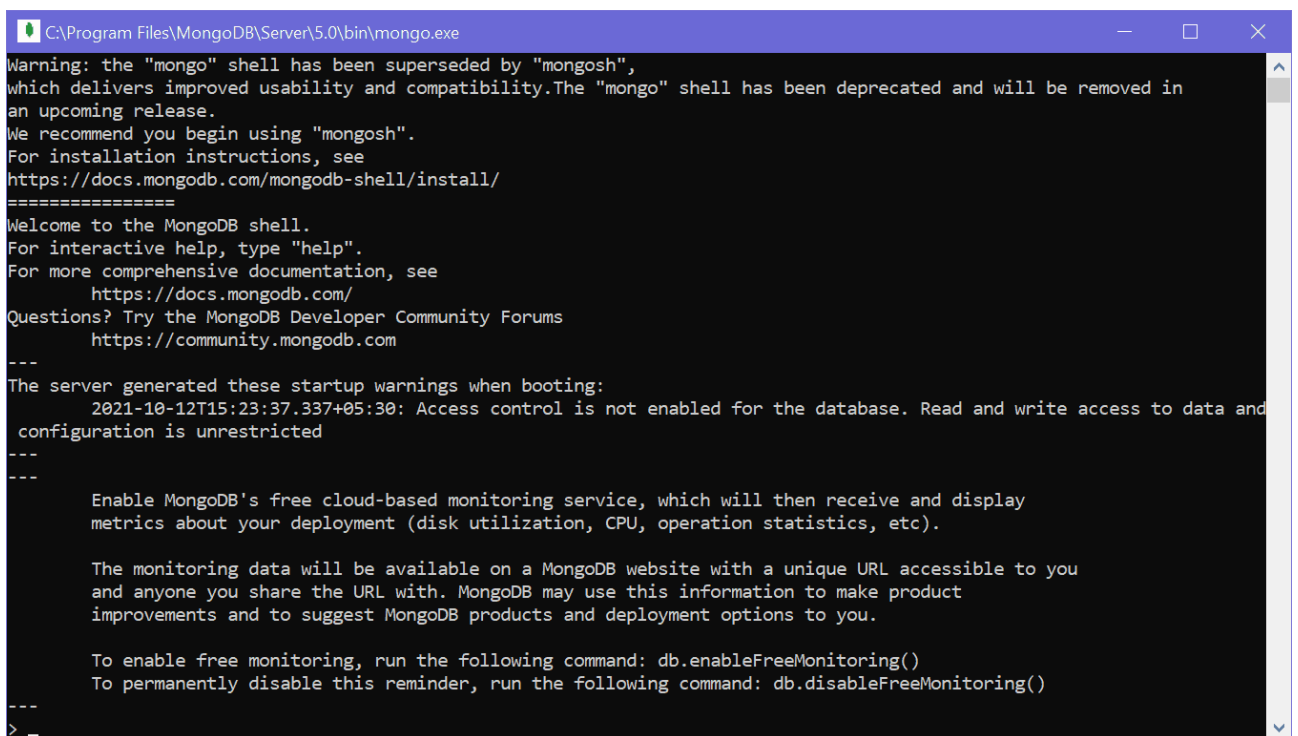
Here, we already installed two clients, MongoDB Shell and MongoDB Compass.

Go to the path where MongoDB was installed. By default it is "C:\Program Files\MongoDB\Server\5.0\bin", as shown below.



MongoDB Installation Folder

Here, you will find mongo.exe, which is MongoDB Shell. Click on mongo.exe to start the client, as shown below.



MongoDB Shell

You can execute the commands here. For example, write "show dbs" and press Enter to display the database it already has, as shown below.

```
C:\Program Files\MongoDB\Server\5.0\bin\mongo.exe

> show dbs
admin    0.000GB
config  0.000GB
local    0.000GB
>
```

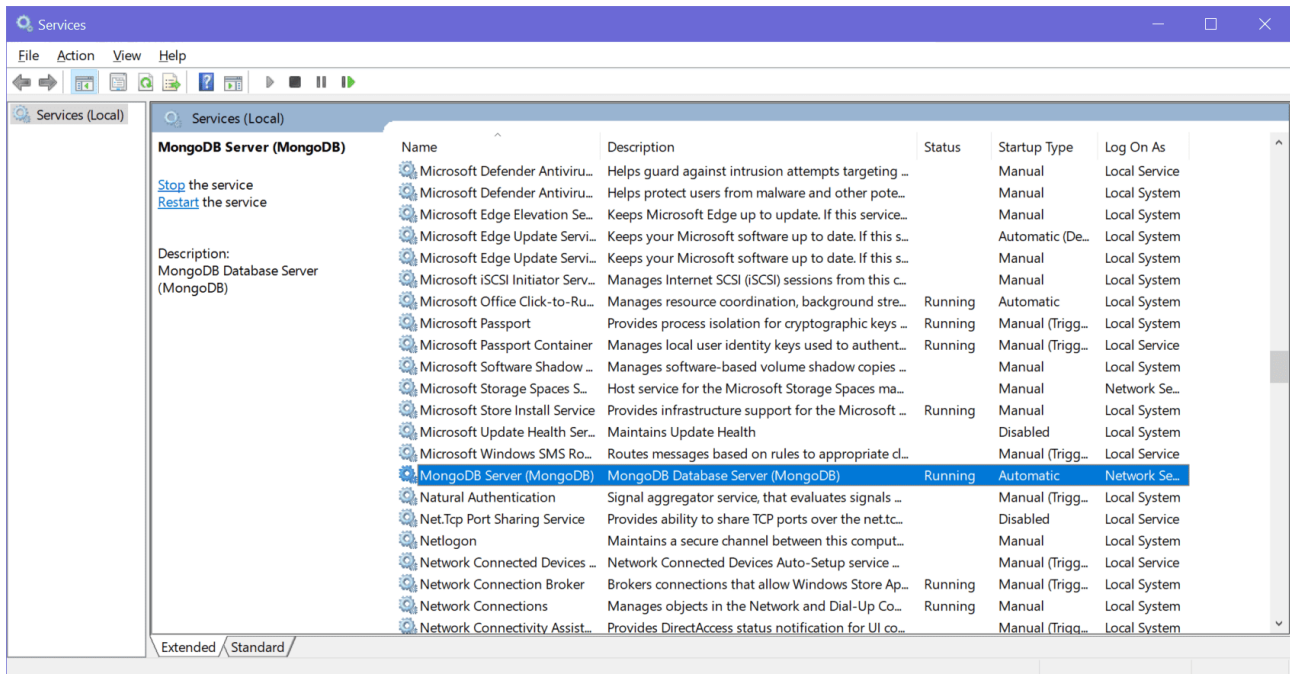
MongoDB Shell Command

Thus, you can install the MongoDB server, clients (MongoDB Shell and Compass), and other features.

MongoDB Server - mongod, Configurations

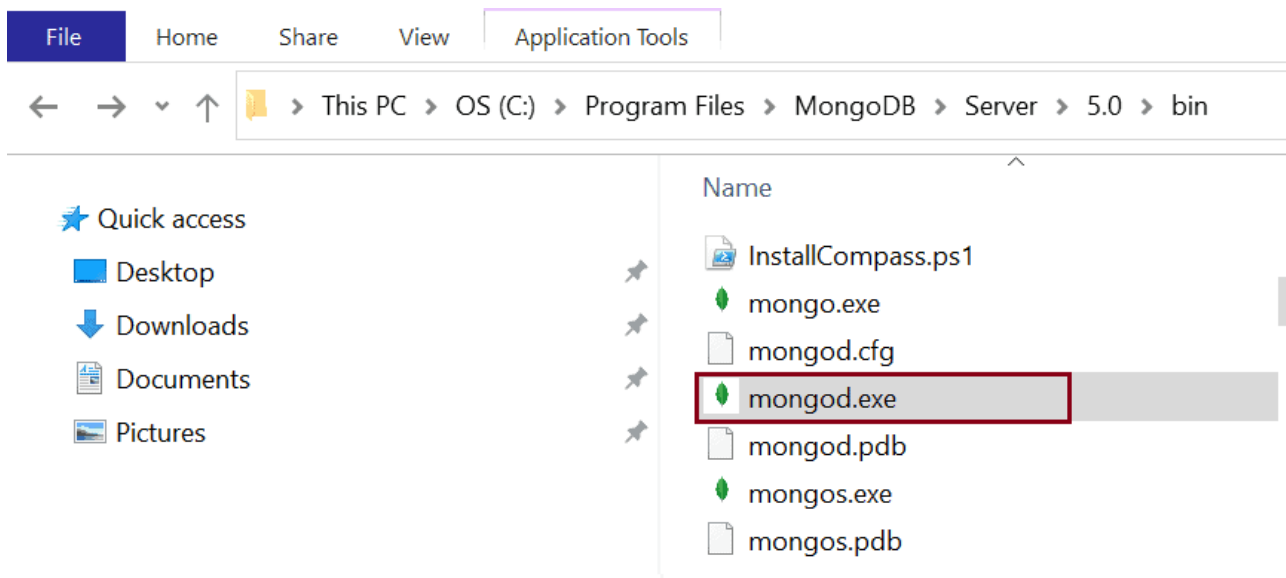
Here you will learn what is MongoDB server, how to run it manually, and how to set various configuration settings for your MongoDB server.

We installed the MongoDB server as a Windows service in the [Install MongoDB](#) chapter. MongoDB windows service is automatically up and running at startup, as shown below.



MongoDB Service on Windows

Internally, the MongoDB service starts the local MongoDB server at the default address `http://127.0.0.1:27017` whenever your machine starts. MongoDB server is installed as `mongod.exe` on Windows in the `{mongodb install folder}/bin` folder, as shown below.



MongoDB Server File - mongod.exe

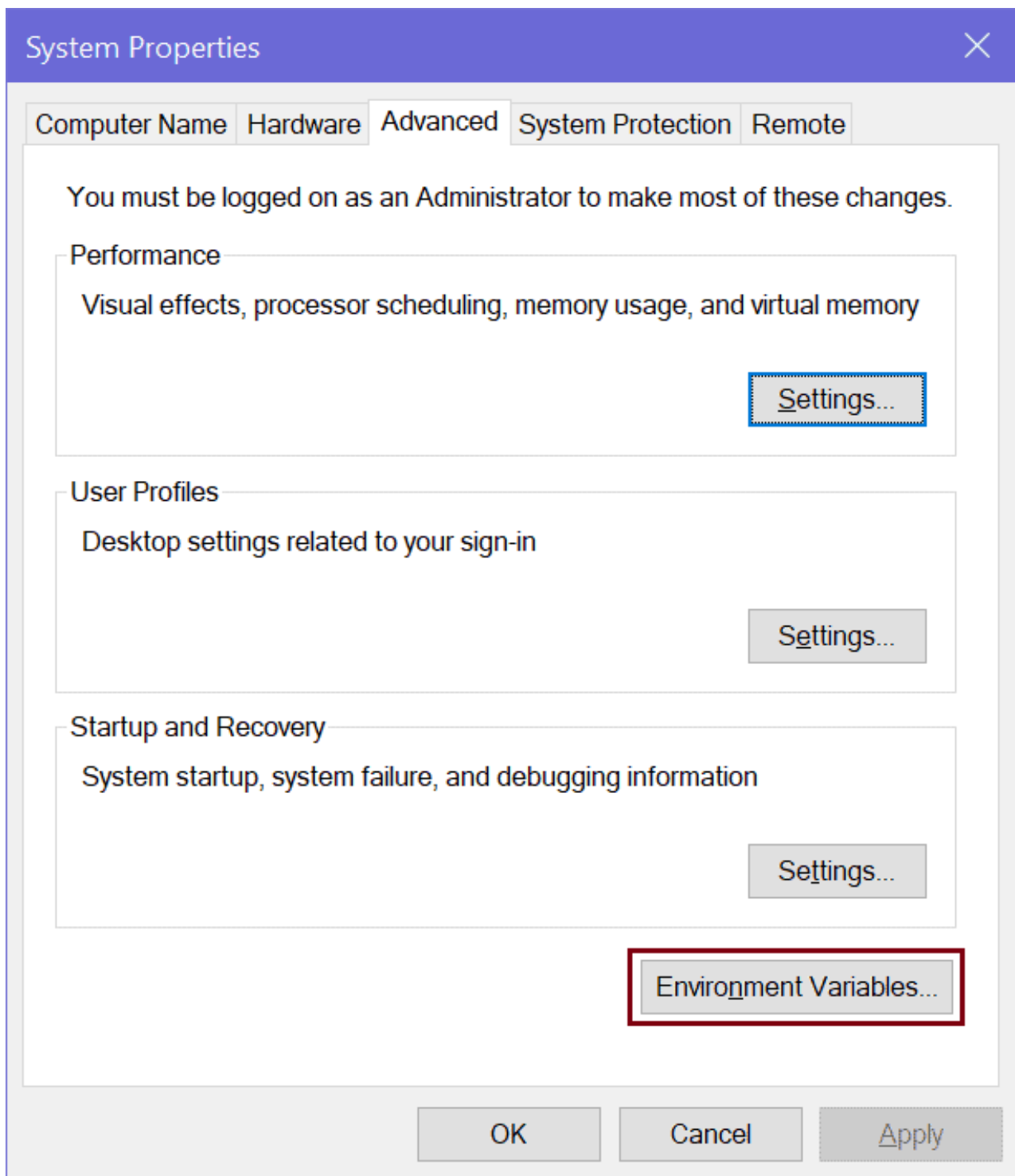
The MongoDB server mongod can be started or stop manually whether you installed it as a service or not.

Run mongod Manually

If you have installed MongoDB as a windows service then stop the service first as we are going to start it manually.

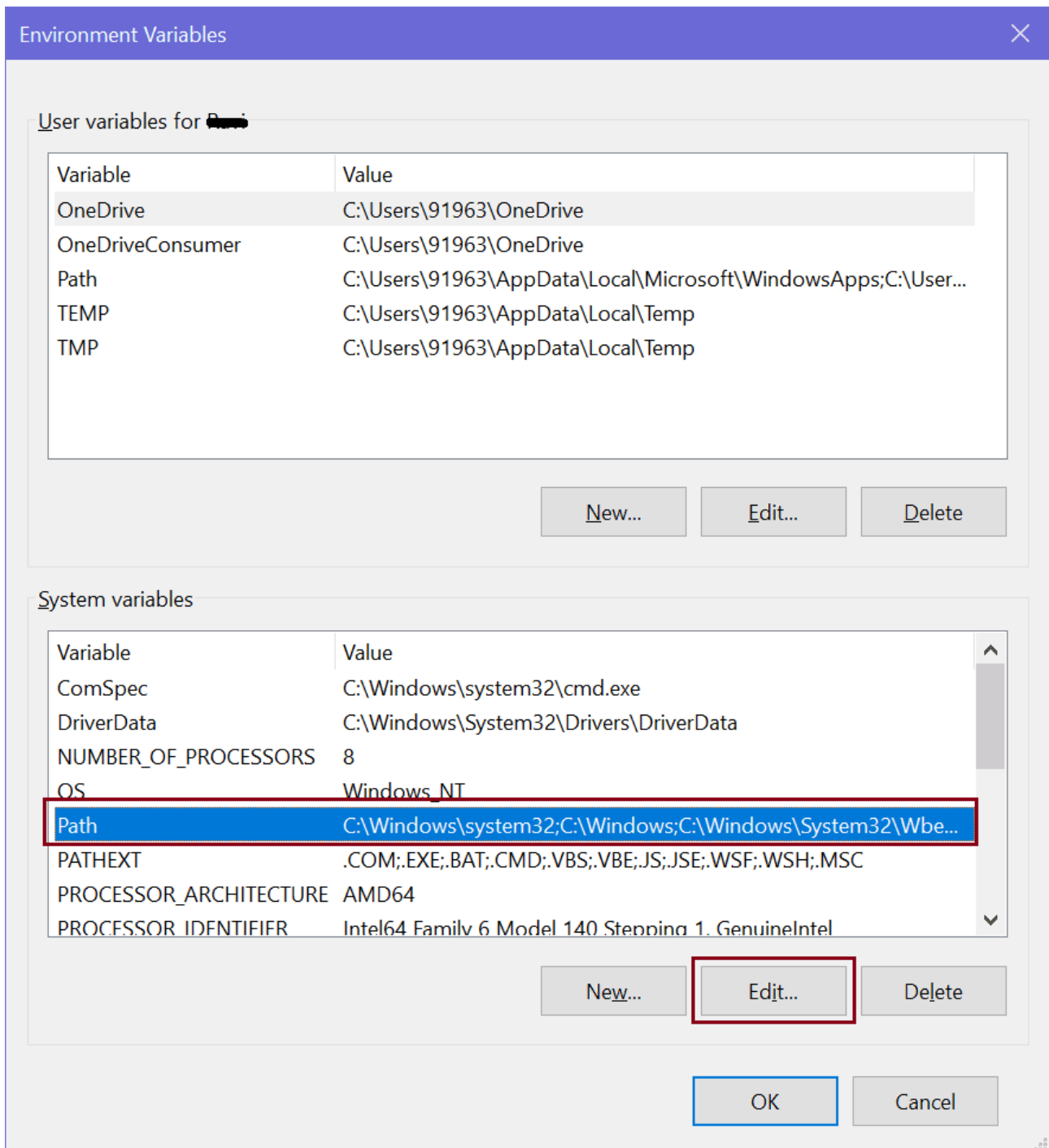
Now, add {mongodb install folder}/bin to your environment variable on your local Windows machine. Search for "Environment" on windows search and click "Edit the system environment variable" to open system properties, as shown below.





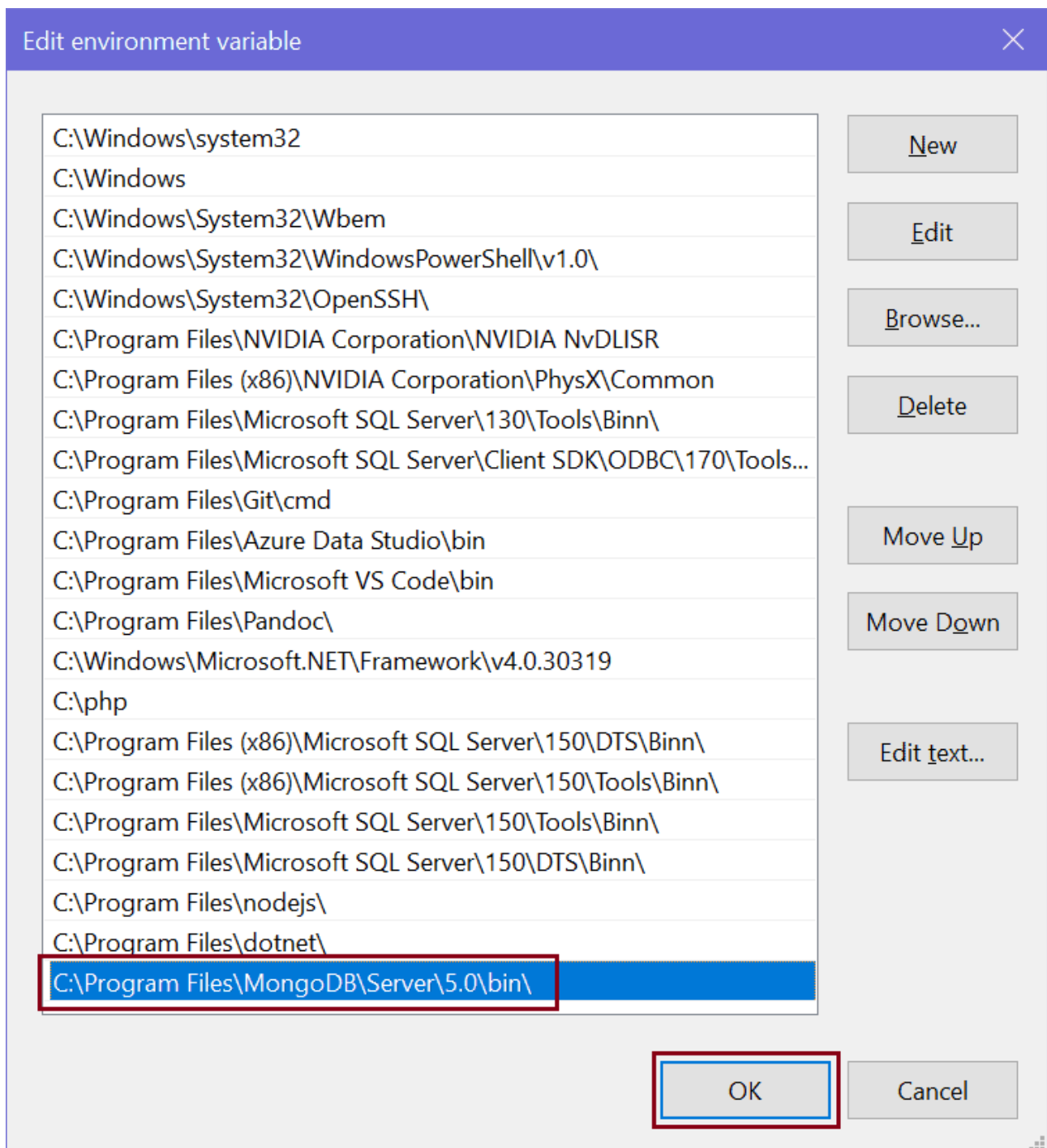
System Properties

Click on "Environment Variables.." to edit path, as shown below.



Environment Variable - Path

Now, select Path in the bottom pane and click on the Edit button. This will open "Edit environment variable" window. Click on the New button to add MongoDB folder path "C:\Program Files\MongoDB\Server\5.0\bin\", as shown below.



Add MongoDB Folder Path

Click OK to close all pop-up windows.

Adding MongoDB folder path to environment variable allow us to run mongod.exe from anywhere on command prompt/terminal.

Now, open a command prompt on Windows or terminal as an administrator and execute the `mongod` on Windows or `sudo mongod` on Mac to run your MongoDB server, as shown below.

```

C:\>mongod
{"t":{"$date":"2021-10-13T14:11:30.055+05:30"},"s":"I", "c":"CONTROL", "id":23285, "ctx":"main","msg":"Automaticall
y disabling TLS 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2021-10-13T14:11:30.410+05:30"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"main","msg":"Initialized
wire specification","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":13},"incomingInternal
Client":{"minWireVersion":0,"maxWireVersion":13},"outgoing":{"minWireVersion":0,"maxWireVersion":13},"isInternalClient"
:true}}}}
{"t":{"$date":"2021-10-13T14:11:30.413+05:30"},"s":"W", "c":"ASIO", "id":22601, "ctx":"main","msg":"No Transport
Layer configured during NetworkInterface startup"}
{"t":{"$date":"2021-10-13T14:11:30.413+05:30"},"s":"I", "c":"NETWORK", "id":4648602, "ctx":"main","msg":"Implicit TCP
FastOpen in use."}
{"t":{"$date":"2021-10-13T14:11:30.415+05:30"},"s":"W", "c":"ASIO", "id":22601, "ctx":"main","msg":"No Transport
Layer configured during NetworkInterface startup"}
{"t":{"$date":"2021-10-13T14:11:30.415+05:30"},"s":"I", "c":"REPL", "id":5123008, "ctx":"main","msg":"Successfully
registered PrimaryOnlyService","attr":{"service":"TenantMigrationDonorService","ns":"config.tenantMigrationDonors"}}
{"t":{"$date":"2021-10-13T14:11:30.415+05:30"},"s":"I", "c":"REPL", "id":5123008, "ctx":"main","msg":"Successfully
registered PrimaryOnlyService","attr":{"service":"TenantMigrationRecipientService","ns":"config.tenantMigrationRecipie
nts"}}
{"t":{"$date":"2021-10-13T14:11:30.416+05:30"},"s":"I", "c":"CONTROL", "id":4615611, "ctx":"initandlisten","msg":"Mon
goDB starting","attr":{"pid":35720,"port":27017,"dbPath":"C:/data/db/","architecture":"64-bit","host":"MyPC"}}
{"t":{"$date":"2021-10-13T14:11:30.416+05:30"},"s":"I", "c":"CONTROL", "id":23398, "ctx":"initandlisten","msg":"Tar
get operating system minimum version","attr":{"targetMinOS":"Windows 7/Windows Server 2008 R2"}}
{"t":{"$date":"2021-10-13T14:11:30.417+05:30"},"s":"I", "c":"CONTROL", "id":23403, "ctx":"initandlisten","msg":"Bui
ld Info","attr":{"buildInfo":{"version":"5.0.3","gitVersion":"657fea5a61a74d7a79df7aff8e4bcf0bc742b748","modules":[],"a
llocator":"tcmalloc","environment":{"distmod":"windows","distarch":"x86_64","target_arch":"x86_64"}}}}
{"t":{"$date":"2021-10-13T14:11:30.417+05:30"},"s":"I", "c":"CONTROL", "id":51765, "ctx":"initandlisten","msg":"Ope
rating System","attr":{"os":{"name":"Microsoft Windows 10","version":"10.0 (build 19043)"}}}
{"t":{"$date":"2021-10-13T14:11:30.417+05:30"},"s":"I", "c":"CONTROL", "id":21951, "ctx":"initandlisten","msg":"Opt
ions set by command line","attr":{"options":{}}}
{"t":{"$date":"2021-10-13T14:11:30.418+05:30"},"s":"I", "c":"STORAGE", "id":22270, "ctx":"initandlisten","msg":"Sto

```

Start MongoDB Server

You can specify various command options with `mongod` command for testing purposes. The following is list of some useful command options.

<code>-h [--help]</code>	Show this usage information
<code>--version</code>	Show version information
<code>-f [--config] arg</code>	Configuration file specifying additional options
<code>--configExpand arg</code>	Process expansion directives in config rest)
<code>--port arg</code>	Specify port number – 27017 by default
<code>--ipv6</code>	Enable IPv6 support (disabled by default)
<code>--listenBacklog arg (=2147483647)</code>	Set socket listen backlog size
<code>--maxConns arg (=1000000)</code>	Max number of simultaneous connections

`--pidfilepath arg`
pidfile (if not set, no
created)

`--timeZoneInfo arg`
zone info directory,

zoneinfo

`-v [ --verbose ] [=arg(=v)]`
(include multiple times

verbosity e.g. `-vvvvv`)

`--quiet`

`--logpath arg`
write to instead of

be a file, not

`--logappend`
instead of

`--logRotate arg`
rotation behavior

`--timeStampFormat arg`
for timestamps in log

`iso8601-utc` or

`--setParameter arg`
configurable parameter

`--bind_ip arg`
list of ip addresses to

localhost by default

`--bind_ip_all`
addresses

`--noauth`
security

`--transitionToAuth`
access control upgrade.

authenticate over outgoing

proceed regardless of

Full path to
pidfile is

Full path to time

e.g. `/usr/share/`

Be more verbose

for more

Quieter output

Log file to send

stdout - has to

directory

Append to logpath

over-writing

Set the log

(rename|reopen)

Desired format

messages. One of

`iso8601-local`

Set a

Comma separated

listen on -

Bind to all ip

Run without

For rolling

Attempt to

connections and

incoming connections authentication.

- `--slowms arg (=100)` profile and console
- `--slowOpSampleRate arg (=1)` ops to include in the console log
- `--profileFilter arg` to control which logged and profiled
- `--auth`
- `--clusterIpSourceAllowlist arg` specification of permitted
- ``__system` access`
 - `--profile arg` 2=all
 - `--cpu` cpu and iowait
 - `--sysinfo` diagnostic system
 - `--noscripting` engine
 - `--notablesan` table scans
 - `--keyFile arg` cluster authentication
 - `--clusterAuthMode arg` mode used for cluster

Alternatives are

`sendKeyFile|sendX509|x509)`

Replication options:

- `--oplogSize arg` MB) for replication op

success. Accept with or without Value of slow for log Fraction of slow profile and Query predicate operations are Run with security Network CIDR origin for 0=off 1=slow, Periodically show utilization Print some information Disable scripting Do not allow Private key for Authentication authentication. (keyFile| Size to use (in

5% of disk space (i.e.	log. default is large is good)
Replica set options:	
--replSet arg	arg is [/]
--enableMajorityReadConcern [=arg(=1)]	(=1) Enables majority
readConcern.	
enableMajorityReadConcern=false is no	longer supported
Sharding options:	
--configsvr	Declare this is a
config db of a	cluster; default
port 27019; default	dir /data/
configdb	
--shardsvr	Declare this is a
shard db of a	cluster; default
port 27018	
Storage options:	
--storageEngine arg	What storage
engine to use - defaults	to wiredTiger if
no data files present	Directory for
--dbpath arg	\data\db\ which
datafiles - defaults to	the current
is C:\data\db\ based on	
working drive	Each database
--directoryperdb	separate
will be stored in a	
directory	
--syncdelay arg (=60)	Seconds between
disk syncs	
--journalCommitInterval arg (=100)	how often to
group/batch commit (ms)	

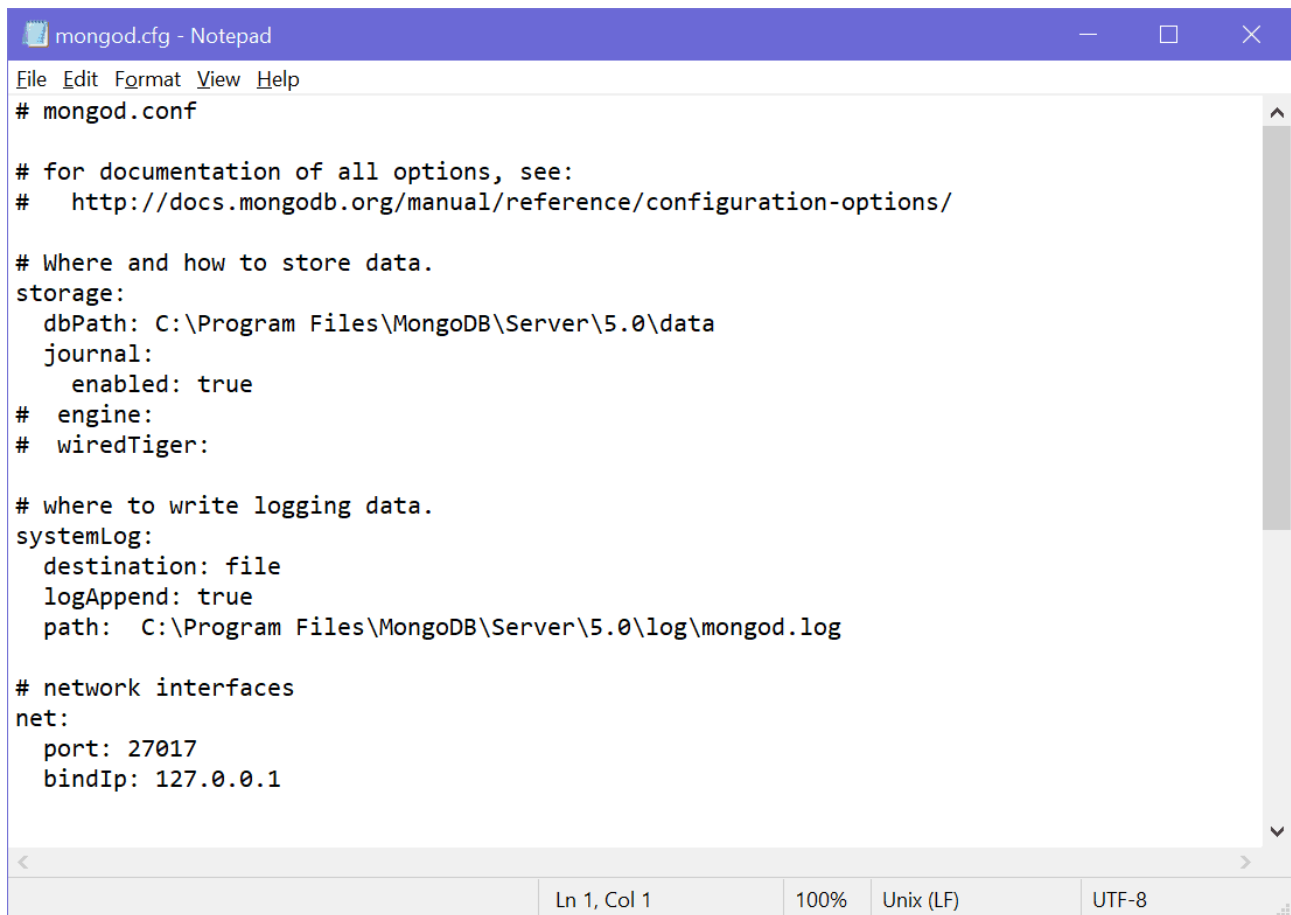
--upgrade needed	Upgrade db if
--repair dbs	Run repair on all
--journal --nojournal journaling (journaling is on by	Enable journaling Disable
bit)	default for 64
--oplogMinRetentionHours arg (=0) hours to preserve in	Minimum number of
Default is 0 (turned off).	the oplog.
allowed (e.g. 1.5 hours)	Fractions are
Free Monitoring Options: --enableFreeMonitoring arg Monitoring	Enable Cloud Free
--freeMonitoringTag arg Monitoring Tags	(on runtime off) Cloud Free
Windows Service Control Manager options:	
--install service	Install Windows
--remove service	Remove Windows
--reinstall service (equivalent	Reinstall Windows
followed by --install)	to --remove
--serviceName arg name	Windows service
--serviceDisplayName arg display name	Windows service
--serviceDescription arg description	Windows service
--serviceUser arg service execution	Account for
--servicePassword arg authenticate	Password used to serviceUser

WiredTiger options:

<code>--wiredTigerCacheSizeGB arg</code>	Maximum amount of
memory to allocate	
	for cache;
Defaults to 1/2 of physical	
	RAM
<code>--zstdDefaultCompressionLevel arg (=6)</code>	Default
compression level for zstandard	
	compressor
<code>--wiredTigerJournalCompressor arg (=snappy)</code>	Use a compressor
for log records	
	[none snappy
zlib zstd]	
<code>--wiredTigerDirectoryForIndexes</code>	Put indexes and
data in different	
	directories
<code>--wiredTigerCollectionBlockCompressor arg (=snappy)</code>	Block compression
algorithm for	
	collection data
[none snappy zlib zstd]	
<code>--wiredTigerIndexPrefixCompression arg (=1)</code>	Use prefix
compression on row-store	
	leaf pages

MongoDB Server Configuration

MongoDB uses the configuration file options to control the behavior of a database. MongoDB server `mongod` runs with the default configurations added in the config file. On Windows, it is `{install directory}\bin\mongod.cfg` file. On MacOS, it is `/usr/local/etc/mongod.conf` or `/opt/homebrew/etc/mongod.conf`. And, on Linux, it is `/etc/mongod.conf` file. MongoDB configuration files use the [YAML](#) format. The following is the `mongod.cfg` file on Windows:



```
File Edit Format View Help
# mongod.conf

# for documentation of all options, see:
#   http://docs.mongodb.org/manual/reference/configuration-options/

# Where and how to store data.
storage:
  dbPath: C:\Program Files\MongoDB\Server\5.0\data
  journal:
    enabled: true
# engine:
# wiredTiger:

# where to write logging data.
systemLog:
  destination: file
  logAppend: true
  path: C:\Program Files\MongoDB\Server\5.0\log\mongod.log

# network interfaces
net:
  port: 27017
  bindIp: 127.0.0.1

Ln 1, Col 1    100%    Unix (LF)    UTF-8
```

mongod.cfg

The following is the sample config file which you may like to use for your local mongod server.

```
systemLog:
  destination: file
  path: "/var/log/mongodb/mongod.log"
  logAppend: true
storage:
  journal:
    enabled: true
processManagement:
  fork: true
net:
  bindIp: 127.0.0.1
  port: 27017
setParameter:
  enableLocalhostAuthBypass: false
```

Key Components of MongoDB Architecture

Below are a few of the common terms used in MongoDB

1. `_id` – This is a field required in every MongoDB document. The `_id` field represents a unique value in the MongoDB document. The `_id` field is like the document's primary key. If you create a new document without an `_id` field, MongoDB will automatically create the field. So for example, if we see the example of the above customer table, MongoDB will add a 24 digit unique identifier to each document in the collection.

<code>_id</code>	Customer ID	CustomerName	OrderID
563479cc8a8a4246bd27d784	11	Guru99	111
563479cc7a8a4246bd47d784	22	Trevor Smith	222
563479cc9a8a4246bd57d784	33	Nicole	333

2. Collection – This is a grouping of MongoDB documents. A collection is the equivalent of a table which is created in any other RDMS such as Oracle or MS SQL. A collection exists within a single database. As seen from the introduction collections don't enforce any sort of structure.
3. Cursor – This is a pointer to the result set of a query. Clients can iterate through a cursor to retrieve results.
4. Database – This is a container for collections like in RDMS wherein it is a container for tables. Each database gets its own set of files on the file system. A MongoDB server can store multiple databases.
5. Document – A record in a MongoDB collection is basically called a document. The document, in turn, will consist of field name and values.
6. Field – A name-value pair in a document. A document has zero or more fields. Fields are analogous to columns in relational databases. The following diagram shows an example of Fields with Key value pairs. So in the example below CustomerID and 11 is one of the key value pair's defined in the document.
7. JSON – This is known as [JavaScript](#) Object Notation. This is a human-readable, plain text format for expressing structured data. JSON is currently supported in many programming languages.

Just a quick note on the key difference between the `_id` field and a normal collection field. The `_id` field is used to uniquely identify the documents in a collection and is automatically added by MongoDB when the collection is created.

Why Use MongoDB?

Below are the few of the reasons as to why one should start using MongoDB

1. Document-oriented – Since MongoDB is a NoSQL type database, instead of having data in a relational type format, it stores the data in documents. This makes MongoDB very flexible and adaptable to real business world situation and requirements.
2. Ad hoc queries – MongoDB supports searching by field, range queries, and regular expression searches. Queries can be made to return specific fields within documents.
3. Indexing – Indexes can be created to improve the performance of searches within MongoDB. Any field in a MongoDB document can be indexed.
4. Replication – MongoDB can provide high availability with replica sets. A replica set consists of two or more mongo DB instances. Each replica set member may act in the role of the primary or secondary replica at any time. The primary replica is the main server which interacts with the client and performs all the read/write operations. The Secondary replicas maintain a copy of the data of the primary using built-in replication. When a primary replica fails, the replica set automatically switches over to the secondary and then it becomes the primary server.
5. Load balancing – MongoDB uses the concept of sharding to scale horizontally by splitting data across multiple MongoDB instances. MongoDB can run over multiple servers, balancing the load and/or duplicating data to keep the system up and running in case of hardware failure.

The use Command

MongoDB **use DATABASE_NAME** is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax

Basic syntax of **use DATABASE** statement is as follows –

```
use DATABASE_NAME
```

Example

If you want to use a database with name **<mydb>**, then **use DATABASE** statement would be as follows –

```
>use mydb
switched to db mydb
```

To check your currently selected database, use the command **db**

```
>db
mydb
```

If you want to check your databases list, use the command **show dbs**.

```
>show dbs
local      0.78125GB
test       0.23012GB
```

Your created database (mydb) is not present in list. To display database, you need to insert at least one document into it.

```
>db.movie.insert({"name":"tutorials point"})
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
```

In MongoDB default database is test. If you didn't create any database, then collections will be stored in test database.

MongoDB Create Database

Use Database method:

There is no create database command in MongoDB. Actually, MongoDB do not provide any command to create database.

It may be look like a weird concept, if you are from traditional SQL background where you need to create a database, table and insert values in the table manually.

Here, in MongoDB you don't need to create a database manually because MongoDB will create it automatically when you save the value into the defined collection at first time.

How and when to create database

If there is no existing database, the following command is used to create a new database.

Syntax:

1. `use DATABASE_NAME`

If the database already exists, it will return the existing database.

Let's take an example to demonstrate how a database is created in [MongoDB](#)

. In the following example, we are going to create a database "sample".

See this example

1. `>use sample`

Switched to db sample

To **check the currently selected database**, use the command db:

1. `>db`

sample

MongoDB Drop Database

The dropDatabase command is used to drop a database. It also deletes the associated data files. It operates on the current database.

Syntax:

1. `db.dropDatabase()`

This syntax will delete the selected database. In the case you have not selected any database, it will delete default "test" database.

MongoDB Drop Database

The dropDatabase command is used to drop a database. It also deletes the associated data files. It operates on the current database.

Syntax:

1. `db.dropDatabase()`

This syntax will delete the selected database. In the case you have not selected any database, it will delete default "test" database.

```
{ "dropped": "sample ", "ok": 1 }
```

Now check the list of databases:

```
1. >show dbs
local 0.078GB
```

The `createCollection()` Method

MongoDB **`db.createCollection(name, options)`** is used to create collection.

Syntax

Basic syntax of **`createCollection()`** command is as follows –

```
db.createCollection(name, options)
```

In the command, **name** is name of collection to be created. **Options** is a document and is used to specify configuration of collection.

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

Options parameter is optional, so you need to specify only the name of the collection. Following is the list of options you can use –

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. If you specify true, you need to specify size parameter also.
autoIndexId	Boolean	(Optional) If true, automatically create index on <code>_id</code> field.s Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. If capped is true, then you need to specify this field also.

max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.
-----	--------	--

While inserting the document, MongoDB first checks size field of capped collection, then it checks max field.

Examples

Basic syntax of **createCollection()** method without options is as follows –

```
>use test
switched to db test
>db.createCollection("mycollection")
{ "ok" : 1 }
>
```

You can check the created collection by using the command **show collections**.

```
>show collections
mycollection
system.indexes
```

The following example shows the syntax of **createCollection()** method with few important options –

```
> db.createCollection("mycol", { capped : true,
autoIndexID : true, size : 6142800, max : 10000 } )
{
"ok" : 0,
"errmsg" : "BSON field 'create.autoIndexID' is an unknown
field.",
"code" : 40415,
"codeName" : "Location40415"
}
>
```

In MongoDB, you don't need to create collection. MongoDB creates collection automatically, when you insert some document.

```
>db.mycollection.insert({"name" : "mycollection"}),
WriteResult({ "nInserted" : 1 })
>show collections
mycol
mycollection
system.indexes
sample
>
```

The drop() Method

MongoDB's **db.collection.drop()** is used to drop a collection from the database.

Syntax

Basic syntax of **drop()** command is as follows –

```
db.COLLECTION_NAME.drop()
```

Example

First, check the available collections into your database **mydb**.

```
>use mydb
switched to db mydb
>show collections
mycol
mycollection
system.indexes
sample
>
```

Now drop the collection with the name **mycollection**.

```
>db.mycollection.drop()
true
>
```

Again check the list of collections into database.

```
>show collections
mycol
system.indexes
sample
>
```

drop() method will return true, if the selected collection is dropped successfully, otherwise it will return false.

Introduction to the mongo shell

The mongo shell is an interactive JavaScript interface to MongoDB. The mongo shell allows you to manage data in MongoDB as well as carry out administrative tasks.

MongoDB client

The mongo shell is a MongoDB client. By default, the mongo shell connects to the **test** database on the MongoDB server and assigns the database connection to the global variable called **db**.

The `db` variable allows you to see the current database:

```
> db  
test
```

Note that when you type a variable or an expression on the command line, the mongo shell will evaluate it and returns the result.

Besides supporting JavaScript syntax, the `mongosh` shell provides you with useful commands that easily interact with the MongoDB database server.

For example, you can use the `show dbs` command to list all the databases on the server:

```
test> show dbs  
admin      41 kB  
config     73.7 kB  
local      81.9 kB
```

Code language: CSS (css)

The output shows three databases on the server.

To switch the current database to another, you use the `use <database>` command. For example, the following command switches the current database to the `bookdb` database:

```
test> use bookdb  
switched to db bookdb
```